

DRONE CAMP

FUNCTIONS / LOOPS / MENUS / FEATURE SWITCHES

DAY 4

ONE POLISHED FLIGHT SYSTEM



FLIGHT
SYSTEM

PYTHON

MAIN.PY



DRONE CAMP

START SCREEN

Build one complete Python flight system

You will develop one `main.py` file in stages. The first checkpoints rebuild functions, indentation, loops, and conditionals. Later checkpoints add reusable helpers, adjustable routines, feature switches, a menu, flips, and geometry puzzles. Complete the required checkpoints in order. Optional power-ups remain available after the required program works.

Stage	What to do	Evidence
Checkpoint 1	Repair a function and verify its indentation.	Console preview
Checkpoint 2	Repeat movement with a for loop.	Loop trace
Checkpoint 3	Choose movement with a conditional.	Branch trace
Checkpoint 4	Create shared helper functions.	DRY code
Routine 2	Complete the mandatory slalom routine.	Working function
Power-ups	Add patrol, finale, menu, switches, and flips.	Polished <code>main.py</code>

Required checkpoints

Complete each code check before moving forward. When a result is wrong, compare indentation, function calls, loop ranges, and branch conditions one at a time.

Optional power-ups

After the mandatory slalom works, add the patrol routine, the finale, polygon calculations, and menu behavior. Each power-up should keep the program readable.

How to use this guide

Stop at every checkpoint. Type the code, predict the result, run a console preview, and explain the control flow before requesting a live flight check.

Trace the route before launch, then test one small change at a time.

[READ](#) → [TRACE](#) → [CODE](#) → [PREVIEW](#) → [FLIGHT CHECK](#)

NORTH STAR

The finished file has one clear structure

FILE MAP - MAIN.PY

```
from easytello import tello

# Settings and feature switches

def pause(...):
    # shared helper

def turn(...):
    # shared helper

def maybe_flip(...):
    # optional feature

def fly_warmup(...):
    # required foundation

def fly_slalom(...):
    # mandatory routine

def fly_patrol(...):
    # optional routine

def fly_finale(...):
    # unlocks four minutes of flight time

def choose_routine():
    # user menu

def run_selected_routine(...):
    # conditional dispatch

def main():
    # connect, check, take off, run, land, close

if __name__ == "__main__":
    main()
```

Why takeoff and landing belong in main()

Each fly_* function should describe movement only. It should not create another drone connection or repeat takeoff and landing. main() performs those shared procedures once. This follows DRY: do not repeat the same procedure in several places when one shared location can control it.

Why helper functions appear first

pause(), turn(), and maybe_flip() describe actions used by more than one routine. The larger functions call these helpers rather than rebuilding the same logic. Python must read a function definition before it reaches a call to that function, so the file moves from general tools toward complete missions.

North-star rule

One connection, one takeoff, one landing, and several reusable movement functions.

Trace the route before launch, then test one small change at a time.

[READ](#) → [TRACE](#) → [CODE](#) → [PREVIEW](#) → [FLIGHT CHECK](#)

CHECKPOINT 01

Repair a function and its indentation

BROKEN EXAMPLE - FIND TWO INDENTATION ERRORS

```
def preview_move(distance):  
    print("Moving forward")  
        print(f"Distance: {distance}")  
  
preview_move(30)
```

What indentation means

The colon after `def preview_move(distance):` begins a function body. Every statement belonging to that function must move inward by the same number of spaces. The function call belongs at file level, so it returns to the left edge. Indentation is Python syntax: it determines which statements belong together.

CORRECTED EXAMPLE - FUNCTION BODY AND FILE LEVEL

```
def preview_move(distance):  
    print("Moving forward")  
    print(f"Distance: {distance}")  
  
preview_move(30)
```

Trace the call

- Python stores the function definition.
- Python reaches `preview_move(30)`.
- The argument 30 enters the parameter distance.
- Both indented print statements run.
- Execution returns to file level.

Checkpoint

Create `preview_turn(angle)` that prints the angle twice: once in a sentence and once by itself.

CHECKPOINT 02

Repeat a command pattern with a for loop

CONSOLE PREVIEW - NO DRONE REQUIRED

```
def preview_square(distance):  
    for side in range(4):  
        print(f"Side {side + 1}")  
        print(f"Forward {distance}")  
        print("Turn 90")  
        print("----")
```

```
preview_square(30)
```

Why the whole block repeats

range(4) creates four cycles. During those cycles, side receives 0, 1, 2, and 3. Every statement indented beneath the for line belongs to the loop and repeats. The blank line after the block returns execution to the function's outer indentation level.

Stage	What to do	Evidence
Cycle 1	side = 0	Prints Side 1
Cycle 2	side = 1	Prints Side 2
Cycle 3	side = 2	Prints Side 3
Cycle 4	side = 3	Prints Side 4

Logic puzzle

A regular polygon completes one full rotation across all of its corners. Divide 360 by the number of sides to calculate each exterior turn. A hexagon needs 60-degree turns. An octagon needs 45-degree turns. Calculate the turn for a nine-sided route before checking with Python.

Checkpoint

Create preview_polygon(sides, distance). Calculate turn_angle = 360 // sides, then preview every side and turn.

Trace the route before launch, then test one small change at a time.

READ → TRACE → CODE → PREVIEW → FLIGHT CHECK

CHECKPOINT 03

Choose a direction with a conditional

LOOP + IF/ELSE

```
def preview_zigzag(steps, distance):
    for step in range(steps):
        print(f"Forward {distance}")

        if step % 2 == 0:
            print("Move left")
        else:
            print("Move right")

preview_zigzag(4, 30)
```

What step % 2 tests

The modulo operator returns a remainder. Even values divide by two with remainder zero, so `step % 2 == 0` is `True` for 0, 2, 4, and so on. Odd values reach the else branch. The loop controls how many steps occur; the conditional controls which sideways movement belongs to each step.

Stage	What to do	Evidence
step = 0	Even branch	Left
step = 1	Odd branch	Right
step = 2	Even branch	Left
step = 3	Odd branch	Right

Logic puzzle

Predict the branch sequence for seven steps. Then change the comparison to `step % 3 == 0`. Which step values reach the first branch? Write the sequence before running the program.

Checkpoint

Create `choose_turn(step)` that returns 'left' for even steps and 'right' for odd steps.

CHECKPOINT 04

Create shared helpers and keep the file DRY

SHARED HELPERS - DEFINED ONCE

```
WAIT_TIME = 1
ENABLE_FLIPS = False

def pause(drone, seconds=WAIT_TIME):
    drone.wait(seconds)

def turn(drone, degrees, clockwise=True):
    if clockwise:
        drone.cw(degrees)
    else:
        drone.ccw(degrees)

    pause(drone)

def maybe_flip(drone, direction="f"):
    if ENABLE_FLIPS:
        drone.flip(direction)
        pause(drone, 2)
```

Default parameter values

seconds=WAIT_TIME and clockwise=True provide default values. pause(drone) uses the shared wait time, while pause(drone, 2) overrides it. turn(drone, 90) rotates clockwise by default. turn(drone, 90, False) selects the counterclockwise branch.

Feature switches

ENABLE_FLIPS is a Boolean setting stored near the top of the file. When it is False, maybe_flip() performs no flip. When it is True, the same function allows the command. This keeps optional behavior in one location instead of scattering extra if statements throughout every routine.

DRY check

Search the file for drone.cw and drone.ccw. After adding turn(), those method calls should remain inside the helper rather than being repeated everywhere.

ROUTINE 01

Build fly_warmup()

REQUIRED FOUNDATION ROUTINE

```
def fly_warmup(
    drone,
    distance,
    height
):
    drone.up(height)
    pause(drone)

    drone.forward(distance)
    pause(drone)

    turn(drone, 90)

    drone.back(distance)
    pause(drone)

    drone.down(height)
    pause(drone)
```

Follow the parameters

distance controls the forward and backward travel. height controls the vertical travel. The same values can be changed at the function call without rewriting the routine. The drone should finish near its starting height and front-to-back position, but it should face a new direction because of the ninety-degree turn.

Stage	What to do	Evidence
Preview	Replace EasyTello methods with print statements.	Expected sequence
Trace	Record height, position, and heading after every command.	Five-row trace
Code	Type the function with consistent indentation.	No syntax errors
Explain	State what each parameter controls.	Two sentences

Required check

The routine contains movement only. takeoff(), land(), and close() belong in main().

Trace the route before launch, then test one small change at a time.

[READ](#) → [TRACE](#) → [CODE](#) → [PREVIEW](#) → [FLIGHT CHECK](#)

ROUTINE 02 - MANDATORY

Build fly_slalom()

MANDATORY ROUTINE - LOOP + CONDITIONAL + PARAMETERS

```
def fly_slalom(
    drone,
    steps,
    forward_distance,
    sideways_distance
):
    for step in range(steps):
        drone.forward(forward_distance)
        pause(drone)

        if step % 2 == 0:
            drone.left(sideways_distance)
        else:
            drone.right(sideways_distance)

    pause(drone)
```

Why this routine matters

fly_slalom() combines the foundation skills in one routine. The function packages a complete behavior. Parameters adjust the route. The loop repeats each slalom step. The conditional alternates left and right movement. The function remains reusable because no distance or repetition count is locked into its body.

Stage	What to do	Evidence
1	Preview fly_slalom(None, 4, 30, 20) using print statements.	Eight actions
2	Write the EasyTello version.	Correct indentation
3	Change steps from 4 to 5 and predict the final sideways direction.	Written prediction
4	Explain why the conditional belongs inside the loop.	One paragraph

Mandatory completion

Do not begin Routine 3 until fly_slalom() previews correctly and the branch sequence is left, right, left, right.

Trace the route before launch, then test one small change at a time.

READ → TRACE → CODE → PREVIEW → FLIGHT CHECK

ROUTINE 03 - OPTIONAL

Build fly_patrol()

OPTIONAL POLYGON ROUTINE

```
def fly_patrol(
    drone,
    sides,
    distance,
    clockwise=True
):
    turn_angle = 360 // sides

    for side in range(sides):
        drone.forward(distance)
        pause(drone)
        turn(
            drone,
            turn_angle,
            clockwise
        )
```

The route calculates its own turn

The function receives the number of sides and calculates the exterior angle. Calling `fly_patrol(drone, 5, 30)` creates a five-sided plan with 72-degree turns. Calling `fly_patrol(drone, 8, 30, False)` creates an eight-sided plan with 45-degree counterclockwise turns.

Think before running

- Which argument creates a triangle?
- Which argument creates a hexagon?
- What turn angle results from twelve sides?
- What changes when clockwise becomes False?
- Which values could create a route too large for the available area?

Optional power-up

Add a label parameter and print the route name, side count, distance, and turn angle before movement begins.

Trace the route before launch, then test one small change at a time.

[READ](#) → [TRACE](#) → [CODE](#) → [PREVIEW](#) → [FLIGHT CHECK](#)

ROUTINE 04 - FLIGHT TIME UNLOCK

Build fly_finale()

FINALE ROUTINE - FOUR-MINUTE FLIGHT CHECK

```
def fly_finale(
    drone,
    distance,
    height,
    use_flip
):
    drone.up(height)
    pause(drone)

    for step in range(4):
        drone.forward(distance)
        turn(
            drone,
            90,
            clockwise=(step % 2 == 0)
        )

    if use_flip:
        maybe_flip(drone, "f")

    drone.down(height)
    pause(drone)
```

A Boolean argument controls the optional feature

use_flip belongs to this function call. ENABLE_FLIPS belongs to the program settings. Both must permit the flip before it occurs: the routine must request it, and the global feature switch must allow it. This creates two separate controls for a movement that should never occur accidentally.

Explain the alternating turns

clockwise=(step % 2 == 0) passes a Boolean expression directly into turn(). Even steps rotate clockwise; odd steps rotate counterclockwise. Trace all four values of step and write the heading changes before requesting a flight check.

Unlock requirement

A correct console preview, a complete trace, and a clear explanation of both flip controls unlock four minutes of live flight time.

Trace the route before launch, then test one small change at a time.

[READ](#) → [TRACE](#) → [CODE](#) → [PREVIEW](#) → [FLIGHT CHECK](#)

FEATURE SWITCHES

Turn program sections on and off

FILE LEVEL - PROGRAM SETTINGS

```
USE_MENU = True

ENABLE_WARMUP = True
ENABLE_SLALOM = True
ENABLE_PATROL = False
ENABLE_FINALE = False
ENABLE_FLIPS = False
```

Settings change behavior without changing routines

These Boolean values act as switches. The routine definitions remain untouched. Changing `ENABLE_PATROL` from `False` to `True` allows the program to call `fly_patrol()`. Changing it back disables that section again. A feature switch is useful when one polished file contains both required and optional behavior.

FUNCTION - RUN EVERY ENABLED ROUTINE

```
def run_enabled_routines(drone):
    if ENABLE_WARMUP:
        fly_warmup(drone, 30, 20)

    if ENABLE_SLALOM:
        fly_slalom(drone, 4, 30, 20)

    if ENABLE_PATROL:
        fly_patrol(drone, 5, 30)

    if ENABLE_FINALE:
        fly_finale(
            drone,
            25,
            20,
            use_flip=True
        )
```

Modular test

Enable one routine at a time during development. Several individually correct routines can still create an unsafe combined route.

Trace the route before launch, then test one small change at a time.

[READ](#) → [TRACE](#) → [CODE](#) → [PREVIEW](#) → [FLIGHT CHECK](#)

USER MENU

Select one routine at runtime

MENU INPUT -> CONDITIONAL DISPATCH

```
def choose_routine():
    print("1 - Warmup")
    print("2 - Slalom")
    print("3 - Patrol")
    print("4 - Finale")

    return input(
        "Choose a routine: "
    ).strip()

def run_selected_routine(
    drone,
    choice
):
    if choice == "1":
        fly_warmup(drone, 30, 20)

    elif choice == "2":
        fly_slalom(drone, 4, 30, 20)

    elif choice == "3":
        fly_patrol(drone, 5, 30)

    elif choice == "4":
        fly_finale(
            drone,
            25,
            20,
            use_flip=True
        )

    else:
        print("Unknown choice.")
        fly_warmup(drone, 20, 15)
```

Why the menu returns text

input() returns a string, so the comparisons use '1', '2', '3', and '4'. run_selected_routine() maps each choice to one function call. The final else branch provides a predictable fallback instead of allowing an unknown input to reach no routine at all.

Menu puzzle

Add choice 5 for a seven-sided patrol. Calculate the required turn before writing the function call.

Trace the route before launch, then test one small change at a time.

[READ](#) → [TRACE](#) → [CODE](#) → [PREVIEW](#) → [FLIGHT CHECK](#)

ASSEMBLY

main() controls the complete program

FUNCTION - ONE CONNECTION, ONE TAKEOFF, ONE LANDING

```
MIN_BATTERY = 40

def main():
    drone = tello.Tello()

    battery = int(
        drone.get_battery()
    )
    print(f"Battery: {battery}%")

    if battery < MIN_BATTERY:
        print(
            "Charge the drone "
            "before flying."
        )
        drone.close()
        return

    if USE_MENU:
        choice = choose_routine()

    drone.takeoff()
    pause(drone, 2)

    if USE_MENU:
        run_selected_routine(
            drone,
            choice
        )
    else:
        run_enabled_routines(drone)

    drone.land()
    drone.close()
```

Follow the decision order

main() creates the EasyTello connection and requests the battery value. A low value closes the connection and returns before takeoff. When the check passes, USE_MENU chooses between one menu-selected routine and the set of enabled routines. Both paths share the same takeoff, landing, and close procedures.

Assembly check

Search main.py for takeoff(), land(), and close(). Each should appear once in the final architecture.

COMPLETE MAIN.PY - PART 1

Settings, helpers, and required routines

FILE LEVEL -> END OF fly_slalom()

```

from easytello import tello

MIN_BATTERY = 40
WAIT_TIME = 1

USE_MENU = True

ENABLE_WARMUP = True
ENABLE_SLALOM = True
ENABLE_PATROL = False
ENABLE_FINALE = False
ENABLE_FLIPS = False

def pause(drone, seconds=WAIT_TIME):
    drone.wait(seconds)

def turn(drone, degrees, clockwise=True):
    if clockwise:
        drone.cw(degrees)
    else:
        drone.ccw(degrees)

    pause(drone)

def maybe_flip(drone, direction="f"):
    if ENABLE_FLIPS:
        drone.flip(direction)
        pause(drone, 2)

def fly_warmup(drone, distance, height):
    drone.up(height)
    pause(drone)

    drone.forward(distance)
    pause(drone)

    turn(drone, 90)

    drone.back(distance)
    pause(drone)

    drone.down(height)
    pause(drone)

def fly_slalom(
    drone,
    steps,
    forward_distance,
    sideways_distance
):
    for step in range(steps):
        drone.forward(forward_distance)
        pause(drone)

        if step % 2 == 0:
            drone.left(sideways_distance)
        else:
            drone.right(sideways_distance)

        pause(drone)

```

This section defines configuration values, shared helpers, the foundation routine, and the mandatory slalom. Every function shown here is complete before the next section begins.

Trace the route before launch, then test one small change at a time.

[READ](#) → [TRACE](#) → [CODE](#) → [PREVIEW](#) → [FLIGHT CHECK](#)

COMPLETE MAIN.PY - PART 2

Optional routines and menu selection

FUNCTION SCOPE - fly_patrol() -> END OF run_selected_routine()

```
def fly_patrol(
    drone,
    sides,
    distance,
    clockwise=True
):
    turn_angle = 360 // sides

    for side in range(sides):
        drone.forward(distance)
        pause(drone)
        turn(
            drone,
            turn_angle,
            clockwise
        )

def fly_finale(
    drone,
    distance,
    height,
    use_flip
):
    drone.up(height)
    pause(drone)

    for step in range(4):
        drone.forward(distance)
        turn(
            drone,
            90,
            clockwise=(step % 2 == 0)
        )

    if use_flip:
        maybe_flip(drone, "f")

    drone.down(height)
    pause(drone)

def choose_routine():
    print("1 - Warmup")
    print("2 - Slalom")
    print("3 - Patrol")
    print("4 - Finale")

    return input(
        "Choose a routine: "
    ).strip()

def run_selected_routine(drone, choice):
    if choice == "1":
        fly_warmup(drone, 30, 20)

    elif choice == "2":
        fly_slalom(drone, 4, 30, 20)

    elif choice == "3":
        fly_patrol(drone, 5, 30)

    elif choice == "4":
        fly_finale(
            drone,
            25,
            20,
            use_flip=True
        )

    else:
        print("Unknown choice.")
        fly_warmup(drone, 20, 15)
```

This section adds the polygon and finale routines, then maps menu choices to named function calls. The page begins and ends at function boundaries.

Trace the route before launch, then test one small change at a time.

[READ](#) → [TRACE](#) → [CODE](#) → [PREVIEW](#) → [FLIGHT CHECK](#)

COMPLETE MAIN.PY - PART 3

Feature switches, main(), and the entry point

FUNCTION SCOPE - run_enabled_routines() -> FILE LEVEL

```
def run_enabled_routines(drone):
    if ENABLE_WARMUP:
        fly_warmup(drone, 30, 20)

    if ENABLE_SLALOM:
        fly_slalom(drone, 4, 30, 20)

    if ENABLE_PATROL:
        fly_patrol(drone, 5, 30)

    if ENABLE_FINALE:
        fly_finale(
            drone,
            25,
            20,
            use_flip=True
        )

def main():
    drone = tello.Tello()

    battery = int(
        drone.get_battery()
    )
    print(f"Battery: {battery}%")

    if battery < MIN_BATTERY:
        print(
            "Charge the drone "
            "before flying."
        )
        drone.close()
        return

    if USE_MENU:
        choice = choose_routine()

    drone.takeoff()
    pause(drone, 2)

    if USE_MENU:
        run_selected_routine(
            drone,
            choice
        )
    else:
        run_enabled_routines(drone)

    drone.land()
    drone.close()

if __name__ == "__main__":
    main()
```

The final section chooses enabled routines or a menu-selected routine. main() owns the shared connection, battery check, takeoff, landing, and close procedures.

FINAL CHECK

Prove the program before flight

Stage	What to do	Evidence
Syntax	Run the file far enough to confirm that Python accepts every definition.	No SyntaxError
Structure	Locate every def line and identify where its indentation ends.	Function map
Loop	Trace all values used by fly_slalom() and fly_finale().	Cycle table
Branches	Predict every left/right and clockwise/counterclockwise decision.	Branch sequence
DRY	Confirm takeoff(), land(), and close() appear in main() only.	Three searches
Preview	Describe the selected route from beginning to end.	Spoken explanation

Common debugging questions

- Is the function defined before it is called?
- Does every indented block use a consistent number of spaces?
- Does the function call supply the expected number of arguments?
- Does the loop repeat the intended number of times?
- Does each conditional compare values of compatible types?
- Is the computer connected to the Tello Wi-Fi network?
- Does each fly_* routine contain movement only?

Four-minute unlock

To unlock the finale flight check, present a correct console preview, trace the four loop cycles, explain the two flip controls, and show that the program contains one shared takeoff and one shared landing.

Final logic puzzle

Create a menu choice for a regular ten-sided patrol. Determine the turn angle, write the function call, and predict the final heading.

Trace the route before launch, then test one small change at a time.